AIM- To generate a PWM signal using a timer on STM32F446RE and control the brightness of an onboard LED by varying the duty cycle.

APPARATUS- STM32 Nucleo-F446RE development board, USB Type-A to Mini-B cable, STM32 CUBE IDE, STM32 CubeMX.

THEORY

Pulse Width Modulation (PWM) is a digital technique used to control analog circuits by generating a square wave signal with a variable duty cycle. The duty cycle represents the percentage of time the signal remains HIGH during one complete period.

Key PWM Parameters:

- **Period (T):** Total time for one complete cycle of the PWM waveform
- **Frequency (f):** Number of cycles per second, $f = \frac{1}{T}$
- **ON Time (T_{ON}):** Duration when signal is HIGH
- **OFF Time (T_{OFF}):** Duration when signal is LOW
- **Duty Cycle (D):** $D = \frac{T_{ON}}{T} \times 100\%$

The average output voltage of a PWM signal is directly proportional to the duty cycle:

$$V_{avg} = V_{max} \times \frac{D}{100}$$

where V_{max} is the peak voltage (3.3V for STM32 GPIO).

STM32F446RE microcontroller features multiple general-purpose timers (TIM2-TIM5, TIM9-TIM14) and advanced timers (TIM1, TIM8) capable of generating PWM signals. Each timer can operate multiple independent PWM channels.

Timer Components:

- **Counter Register (CNT):** 16-bit or 32-bit up/down counter
- **Prescaler Register (PSC):** Divides input clock frequency
- **Auto-Reload Register (ARR):** Defines maximum counter value (period)
- **Capture/Compare Register (CCR):** Defines duty cycle threshold
- **Output Compare Mode:** Generates output based on CNT vs CCR comparison

PWM Mode 1 Operation:

In PWM Mode 1, the timer output channel is:

- **HIGH** when CNT < CCR
- **LOW** when CNT ≥ CCR

The counter increments from 0 to ARR, then resets to 0, creating a periodic waveform.

PWM Frequency Calculation:

The PWM frequency is determined by:

$$f_{PWM} = \frac{f_{CLK\_INT}}{(PSC + 1) \times (ARR + 1)}$$

where:

- f_{CLK_INT} = Timer input clock frequency (typically APB1 or APB2 bus clock)
- PSC = Prescaler value
- ARR = Auto-reload value

Duty Cycle Calculation:

$$\text{Duty Cycle (\%)} = \frac{CCR}{ARR + 1} \times 100$$

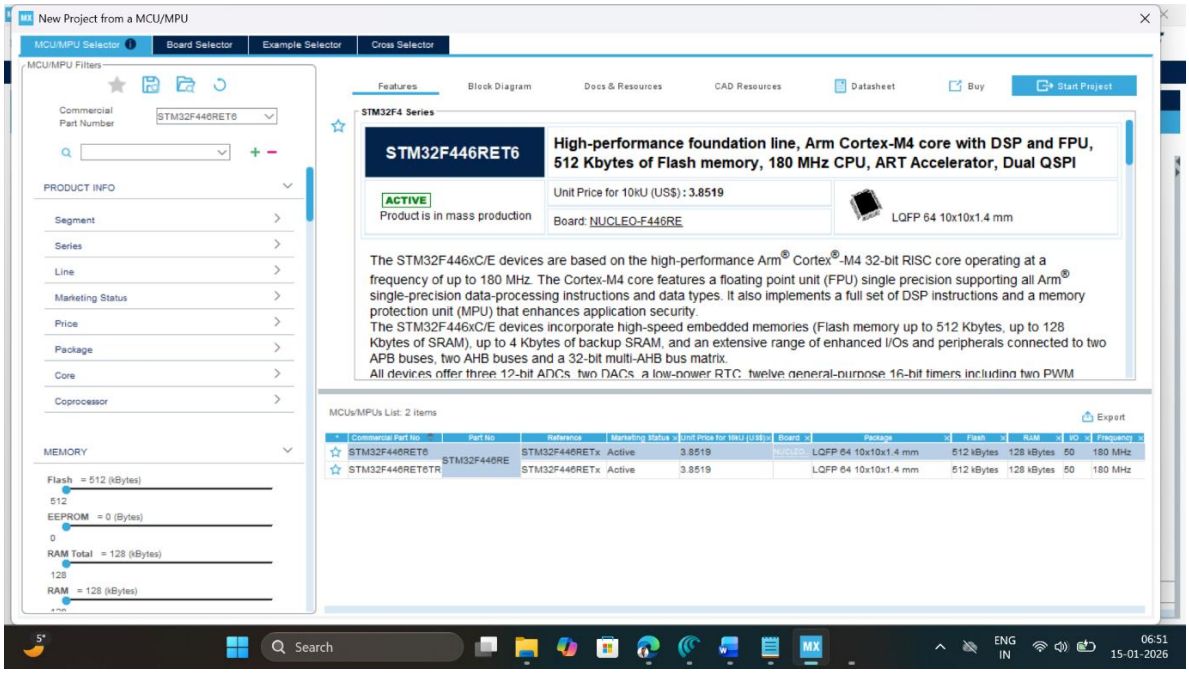
Example Configuration:

To generate 1 kHz PWM with 50% duty cycle using TIM2 (90 MHz clock):

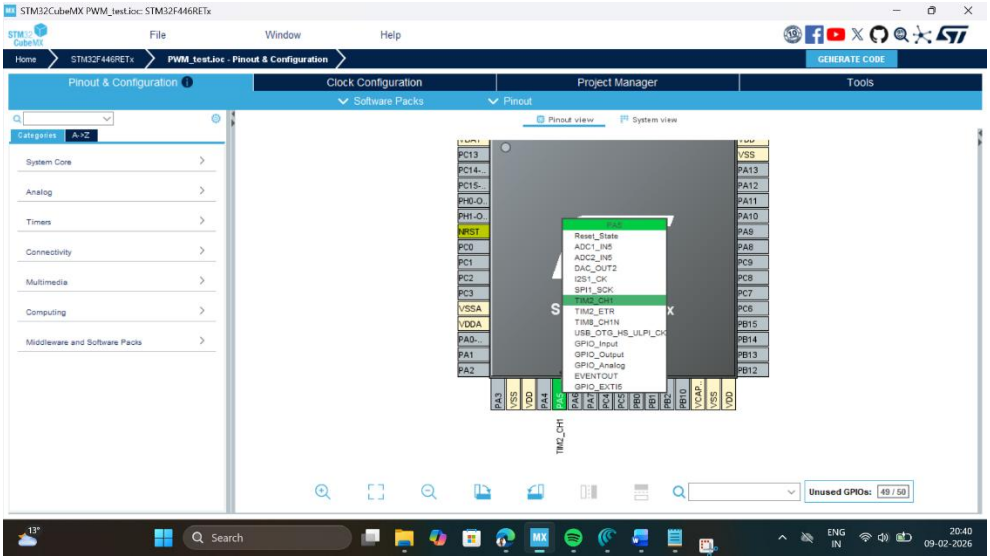
- PSC = 89 → Timer clock = $\frac{90\ \text{MHz}}{90} = 1\ \text{MHz}$
- ARR = 999 → Period = $\frac{1\ \text{MHz}}{1000} = 1\ \text{kHz}$
- CCR = 500 → Duty cycle = $\frac{500}{1000} \times 100 = 50\%$

PROCEDURE

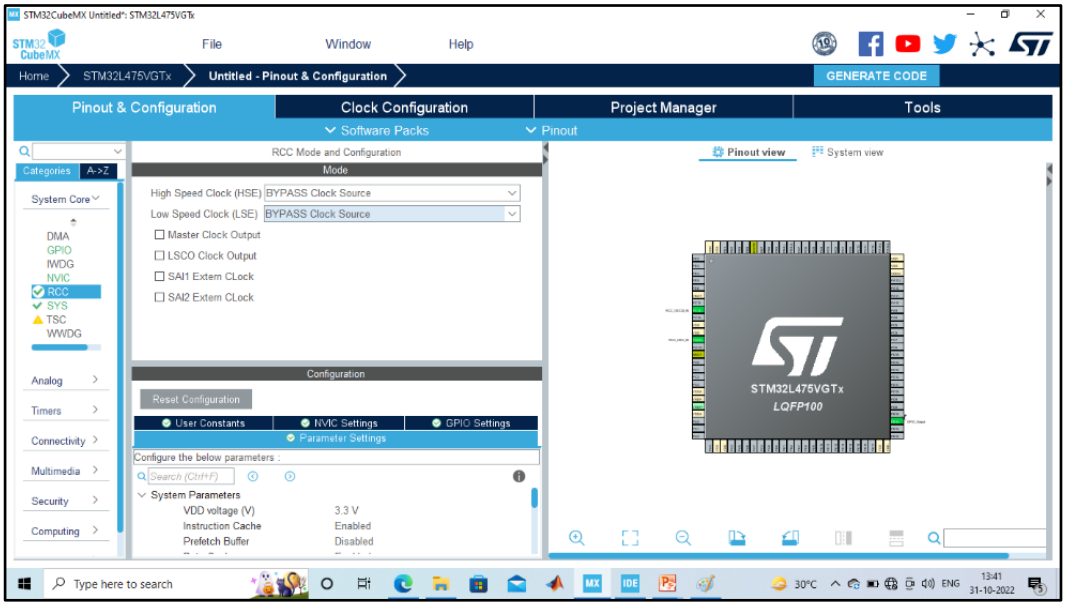
1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.



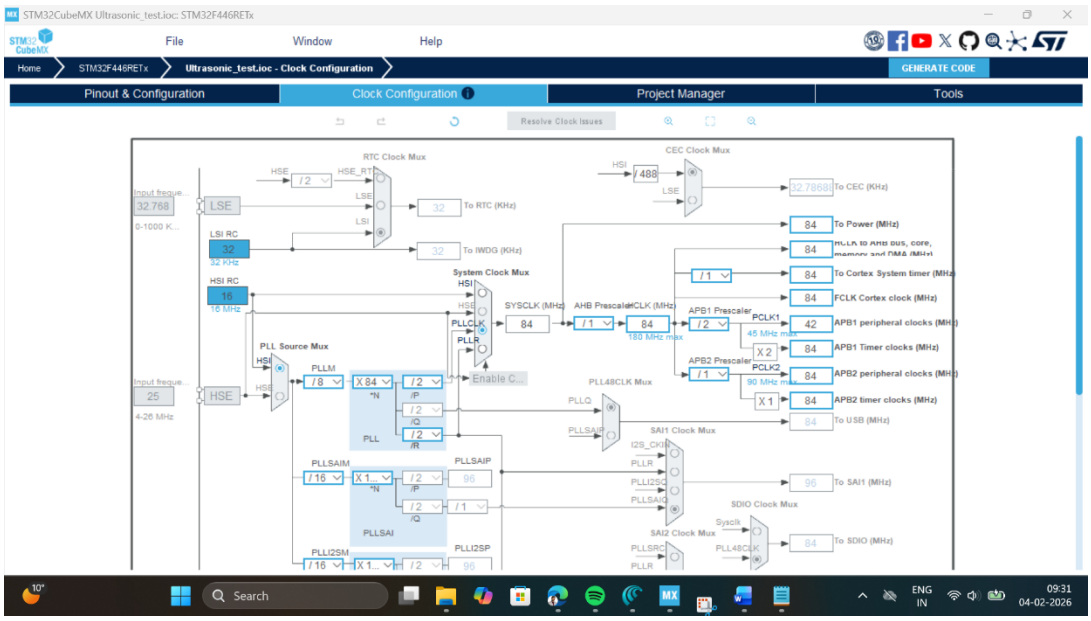
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L446RE.
3. In system Core select GPIO and right click on pin PA5 to configure it as TIM2_CH1.



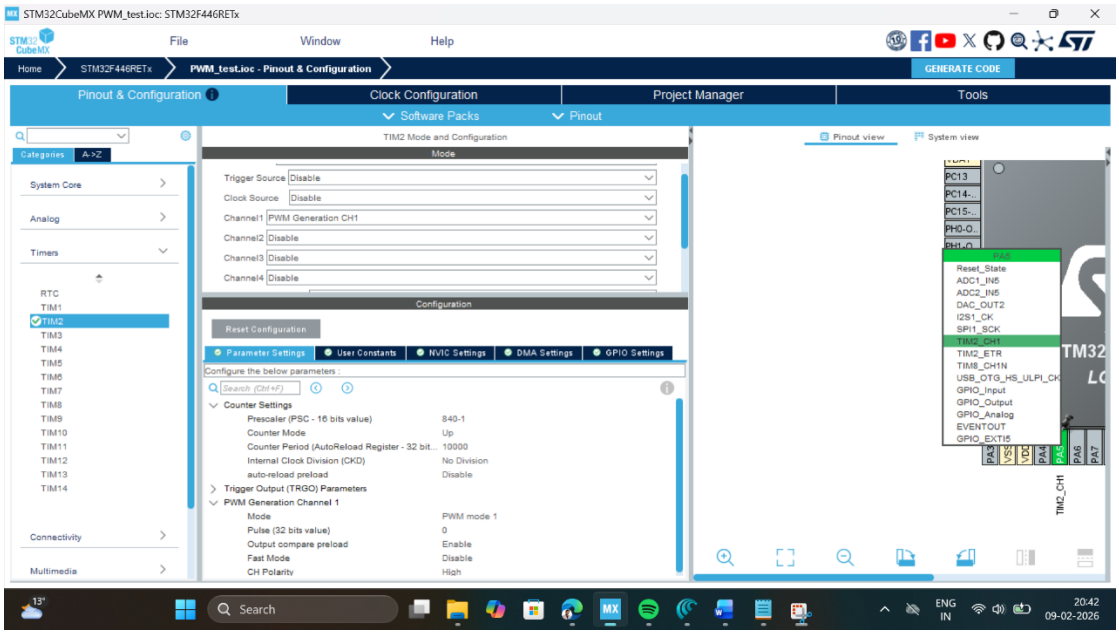
4. **Clock Selection**: Select the internal clock by clicking RCC and selecting BYPASS clock source



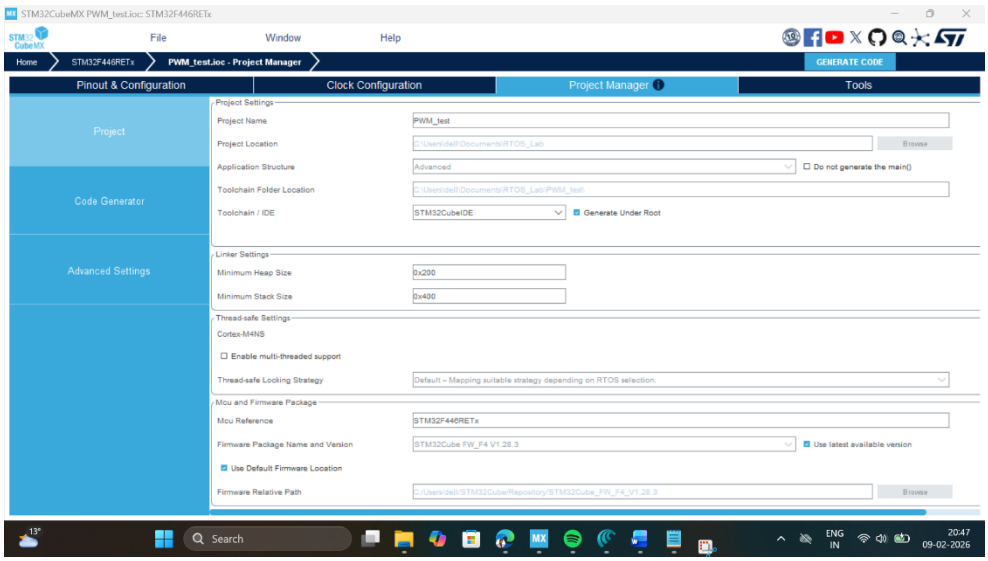
5. **Clock Configuration:** Configure clock as shown below to get maximum internal clock frequency of 84MHz.



6. Set the parameters for TIM2 as follows: Pre-scaler = 840-1, Counter mode = UP, ARR = 1000, so that the generate 100Hz output which will create 100cycles/second.



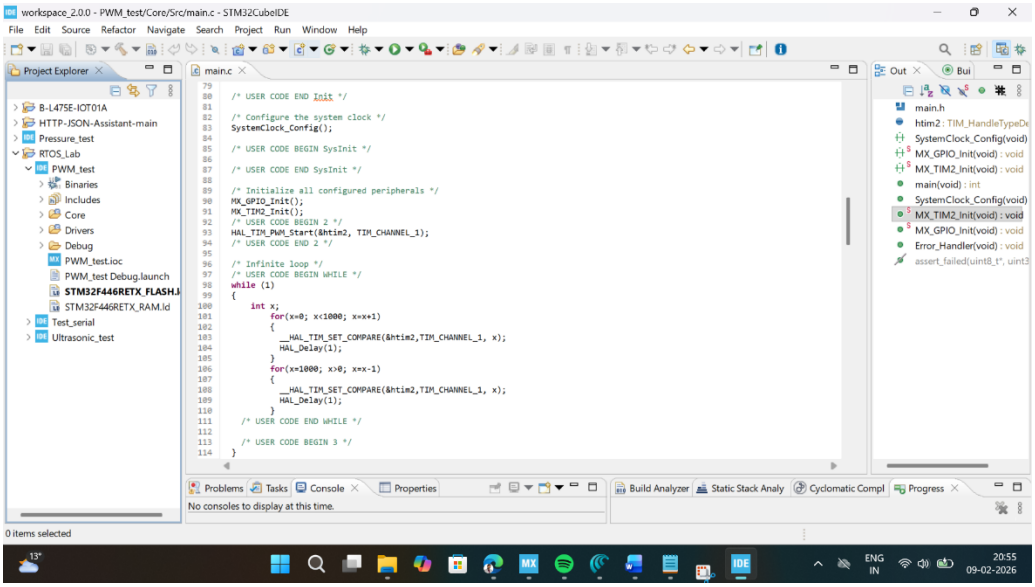
7. Write Project name and select IDE.



8. Click on generate code.
9. Click on open project.
10. Click 'OK' on the generated popup showing "successfully imported the project on the workspace" and the project will be successfully added to the cube IDE.
11. Open the main source code on Cube IDE by clicking on **main.c**
12. To generate bin file and hex file, right click on project and click properties.
13. Expand C/C++ Build, click on setting, click on MCU post build outputs and select convert to binary file as well as convert to hex file and click "Apply and Close ".
14. After configuration write following instructions in the **main.c** file and compile the program and ensure there will be zero error after compiling.

```
/* USER CODE BEGIN 2 */
  HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
  /* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
  while (1)
  {
    int x;
    for(x=0; x<1000; x=x+1)
    {
      __HAL_TIM_SET_COMPARE(&htim2,TIM_CHANNEL_1, x);
      HAL_Delay(1);
    }
    for(x=1000; x>0; x=x-1)
    {
      __HAL_TIM_SET_COMPARE(&htim2,TIM_CHANNEL_1, x);
      HAL_Delay(1);
    }
    /* USER CODE END WHILE */
```



15. To run this program, connect the board with USB cable, click on build, click 'ok' and click 'switch' on popups.
16. Now you are in debugging mode, click 'Resume' to execute the program.

OBSERVATION:

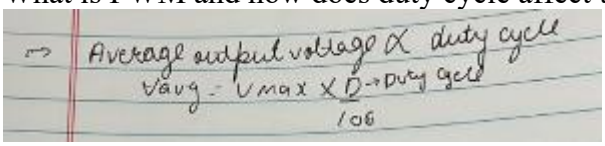
CCR Value	Duty Cycle (%)	Brightness Level	Observation
0	0	Led is off	Led is off
250	25	Very dim	Barely visible
500	50	Dim	Noticeable
750	75	Visible	Clearly lit
1000	100	Bright	Fully on

RESULT

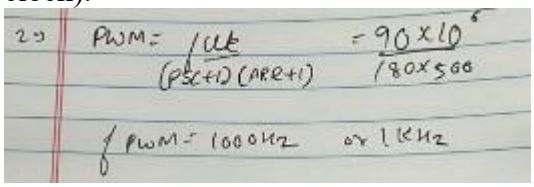
PWM signal was successfully generated using TIM2 on STM32F446RE, and the onboard LED brightness was controlled by varying the duty cycle from 0% to 100%. The fade-in and fade-out effect demonstrated the relationship between PWM duty cycle and LED brightness.

Reflection Summary

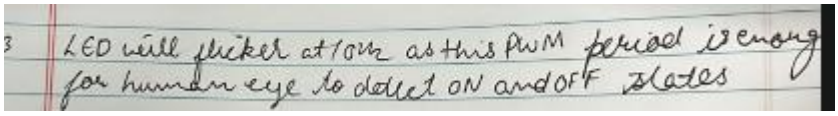
1. What is PWM and how does duty cycle affect the average output voltage?



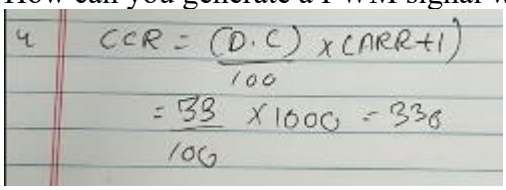
2. Calculate the PWM frequency if PSC = 179 and ARR = 499 (assuming 90 MHz timer clock).



3. What would happen if PWM frequency is reduced to 10 Hz? Would the LED appear to flicker?



4. How can you generate a PWM signal with 33% duty cycle using ARR = 999?



5. How would you modify the code to make the LED fade faster?